

Detailed Grading Rubric (100 pts)

Please note that, to avoid data leakage, cleaning and feature engineering should ideally be performed within a pipeline.

General Analysis of the Business Problem (2.5 pts)

- What is the problem?
- Why is it important to address it?
- How do you intend to address it?
- Etc.

Data Collection (2.5 pts)

- Installation and import of relevant packages and libraries (1.25 pts)
- Data loading (1.25 pts)
 - Merge and concatenate if necessary
 - Creation of subsets if necessary

*Data Cleaning (5 * 2 pts)*

As part of data cleaning, you need at least 5 cleaning steps in the pipeline. You can try:

- Handling missing values
- Removing redundant observations & outliers
- Recoding variables – dates, percentages, text (e.g., “10 years” -> 10)
- Adjusting data types
- Renaming columns (avoiding duplicated columns)
- Encoding variables (one-hot encoding, label encoding, etc.)
- Transforming variable scale (e.g., log transformation for skewed variables, standardization, etc.)
- Etc.

*Feature Engineering (10 * 2 pts)*

As part of feature engineering, you need to create at least 10 steps in the pipeline. For example:

- *Data Sanitization*
 - Remove features based on intuition or business understanding
 - Remove redundant features
 - Remove features that have a high ratio of missing values
 - Remove features that do not vary (i.e., constant values)
 - Remove features that have little variance (e.g., 90% of values are the same)
 - Remove features with a low correlation to the target variable
 - Remove categorical variables with high cardinality

- Adversarial validation (i.e., drop features that cause Data Drift or "Train-Test Mismatch").
- *Creative Engineering (Information Gain)*
 - Create interaction features (e.g., quadratic terms, a feature that describes both card type and address)
 - Transform date time into different scales (e.g., seconds, minutes, hours, days, months, years, etc.)
 - Create clusters
 - Create aggregations (e.g., average and standard deviation of transaction amount per day or per cluster, number of unique values of a feature)
 - Create composite features (e.g., weighted sum, weighted averages)
 - Create ratios (e.g., debt to income)
- *Final Selection*
 - Remove collinear features (e.g., using correlation matrix or VIF)
 - Select K Best, Mutual Information
 - Recursive feature elimination (drop features if the AUC increases)
 - Remove features that have a low consistency over time (e.g., keep features if AUCs early and late are both above 0.5)
 - Dimension reduction (e.g., PCA), if necessary

Data Visualization (5 * 1 pts)

As part of data visualization, you need at least 5 plots. You can try:

- *Univariate plots – how is the target variable distributed?*
 - Plot the target variable (e.g., number of bad vs good loans)
 - Plot the distribution of features (e.g., histograms and density plots)
- *Bivariate plots – how does feature X relate to the target variable?*
 - Plot the target variable across values of categorical variables (e.g., the number of fraudulent transactions per product code, the number of default loans per loan term).
 - Plot histograms of the continuous variables per value of the target variable (e.g., the histogram of transaction amount for fraudulent and non-fraudulent transactions, the histogram of loan amount for default and non-default loans).
 - Plot correlation between the target variable and features (e.g., heatmap with correlation)
- *Multivariate plots – how do features X and Y together affect the target variable?*
 - For example, scatter plots, pair plots, etc., using the target variable as the hue.

Models (28.75 pts)

- Add a resampling step in the pipeline to handle the imbalanced data issue **(2 pts)**

There are different options to handle imbalanced datasets. For example, you can try:

- Class weighting

- Over-sampling
- Under-sampling
- Train 4 different ML models. Each pipeline would have a different ML model (15 pts)
 - Choose diverse models to create a good mix (**4 * 2 pts**)
 - Models that are not sensitive to feature scaling (e.g., Decision Trees)
 - Non-parametric models (i.e., do not assume a certain data distribution) (e.g., Nearest Neighbor)
 - Models that are robust to overfitting (e.g., Random Forests)
 - Models with different representations
 - Probabilistic models (e.g., Naïve Bayes)
 - Instance-based models (e.g., Nearest Neighbor)
 - Symbolic models (e.g., Decision Trees)
 - Numerical models (e.g., Logistic Regression)
 - Etc.
 - Ensemble learning models
 - Bagging (e.g., Random Forest, Extra Trees)
 - Boosting (e.g., Adaboost, Gradient Boosting)
 - Etc.
 - Neural network models
 - Etc.
 - Choose one scoring metric (e.g., accuracy, precision, recall, AUC, etc.) (**1 pt**)
 - Split the data into train and test data (**1 pt**)
 - Report the results of K-fold validation (**2 pts**)
 - You can plot the performance using box plots
 - Report the results on the test data across 4 scoring metrics (e.g., accuracy, precision, recall, AUC, etc.) (**4 pts**)
 - You can report the results in a table
- Choose the pipeline with the best model (**2 pts**)
 - Explain your decision (e.g., which metric did you use)
 - Does your best model overfit? If yes, how are you addressing it in the project
 - Compare training and test performance to detect overfitting
 - If the model overfits, it will be better to address it or choose another model
 - Common approaches to addressing overfitting include regularization, removing outliers, feature scale transformation, reducing the number of features, or using an algorithm robust to overfitting or outliers
- Fine-tune the best pipeline (**8.75 pts**)
 - As part of model fine-tuning, you can try different configurations of parameters to find the optimal one. You can do it using Grid Search.
 - Vary at least 4 parameters in the grid search (**5 pts**)
 - Save the fine-tuned pipeline (**1.25 pts**)
 - Report the results of the fine-tuned pipeline on the test data across 4 scoring metrics (**2.5 pts**)
 - After comparing with the study that you summarized in the 1st milestone:
 - If the model performance is too low, possible causes are:

- Insufficient sample size for training
- Mispecified models (i.e., lack of crucial features)
- Multicollinearity (i.e., highly correlated features)
- Choice of relationship (e.g., linear vs quadratic)
- Etc.
- If the model performance is too high, possible causes are:
 - Imbalanced dataset
 - Misleading choice of the scoring metric
 - One of the features is highly correlated with the target variable
 - Not enough observations in both classes (e.g., only 2 loans are bad, and the remaining 900 loans are good)
 - Etc.
- If the model performance is not too low or too high, then your feature engineering looks good.

Finalize Model (6.25 pts)

- Rank the feature for importance **(2.5 pts)**
- Local explainability **(2.5 pts)**
- Save the SHAP Explainer **(1.25 pts)**

Deployment (15 pts)

- AWS deployment **(5 pts)**
- Create Streamlit Web App **(10 pts)**
 - Prediction **(5 pts)**
 - SHAP plot **(5 pts)**

Conclusion – Executive Summary (10 pts)

- Highlight objective (i.e., business problem)
- Provide business-relevant results metrics
- Explain how the model would be used
- Indicate key drivers of the prediction (top features)
- Explain the business impact (i.e., how the results solve the business problem)
- Discuss risks and limitations (i.e., weaknesses) of the model
- Provide recommendations to executives (e.g., suggestions about validation, improvement, or integration of the model)

Deliverables

- Well-organized notebook – Technical Report
- Executive Summary
- Demo of the Streamlit Web App (screenshots or short video)